Names of the AIs used:

Claude - Sonnet 5.5 Medium

Deepseek - Free

Access method:

Simply entered each name into a google search query individually and used their respective websites.

Prompt used (Included the puzzle1.txt and solution1.txt files):

```
Write a program in C++ following the instructions below.

Given a partially completed Sudoku puzzle, find the/a solution;
if no solution exists, prove that. You will be given five files
(puzzle1.txt through puzzle5.txt), each containing a Sudoku
puzzle. Digits already filled in have been placed, and an
underscore character (_) is used to indicate a blank square.
The solution may not be unique; if there is more than one
solution to the
puzzle, your program should print them all out.

The method: implement a brute-force search, write a recursive
object-oriented program to find the solution with the
depth-first search with backtracking approach.

The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is
found, “No solution found” printed out.
To help you test the program, solution1.txt is a solution to
puzzle1.txt
```

Claude Code:

```
//claude.cpp contained in file Joshua_Louis_Assignment_4
//C++ program that solves sudoku puzzles.
//Creation Date: 11:05AM, Thursday, October 15, 2026
//Inputs: direct inputs with bash command "./claude puzzle1.txt" or just run "./claude"
for all 5 puzzles
/*Outputs: The program outputs the puzzle's file, puzzle itself,
    each solution, and solutions found for every input.*/
//Collaborators: Claude - Sonnet 5.5 Medium
//Note: None of the code below is original/human-made, it was all generated by Claude -
Sonnet 5.5 Medium
//Author: Joshua Louis

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class Sudoku {
public:
    static const int N = 9;

    explicit Sudoku(const std::string &filename)
        : name(filename), solutionCount(0), loaded(false) {
        for (int r = 0; r < N; r++)
            for (int c = 0; c < N; c++) grid[r][c] = 0;
        loaded = load();
        for (int r = 0; r < N; r++)
            for (int c = 0; c < N; c++) original[r][c] = grid[r][c];  // keep original
for printing
    }

    bool isLoaded() const { return loaded; }

    // Prints the puzzle, then solves it, printing every solution found.
    void run() {
        std::cout << "Puzzle file: " << name << "\n\n";
        std::cout << "Puzzle:\n";
        print(original);
        std::cout << "\n";
```

```
        if (!validStart()) {
            std::cout << "No solution found\n\n";
            return;
        }

        solve(0);

        if (solutionCount == 0)
            std::cout << "No solution found\n";
        else
            std::cout << "Solutions found: " << solutionCount << "\n";
        std::cout << "\n";
    }

private:
    std::string name;
    int grid[N][N];
    int original[N][N];
    int solutionCount;
    bool loaded;

    bool load() {
        std::ifstream in(name.c_str());
        if (!in) {
            std::cerr << "Error: cannot open " << name << "\n";
            return false;
        }
        for (int r = 0; r < N; r++) {
            for (int c = 0; c < N; c++) {
                char ch;
                if (!(in >> ch)) {
                    std::cerr << "Error: " << name << " has too few cells\n";
                    return false;
                }
                if (ch == '_' || ch == '0' || ch == '.')
                    grid[r][c] = 0;
                else if (ch >= '1' && ch <= '9')
                    grid[r][c] = ch - '0';
                else {
                    std::cerr << "Error: bad character '" << ch << "' in " << name <<
"\n";
```

```
                return false;
            }
        }
    }
    return true;
}

// Can value v legally sit at (r, c)? (ignores the cell's own content)
bool canPlace(int r, int c, int v) const {
    for (int i = 0; i < N; i++) {
        if (i != c && grid[r][i] == v) return false;
        if (i != r && grid[i][c] == v) return false;
    }
    int br = (r / 3) * 3, bc = (c / 3) * 3;
    for (int i = br; i < br + 3; i++)
        for (int j = bc; j < bc + 3; j++)
            if ((i != r || j != c) && grid[i][j] == v) return false;
    return true;
}

// Check the given clues don't already conflict with each other.
bool validStart() const {
    for (int r = 0; r < N; r++)
        for (int c = 0; c < N; c++)
            if (grid[r][c] != 0 && !canPlace(r, c, grid[r][c])) return false;
    return true;
}

// Recursive DFS over cells in row-major order; pos = 0..80.
void solve(int pos) {
    if (pos == N * N) {  // every cell filled: a solution
        solutionCount++;
        std::cout << "Solution " << solutionCount << ":\n";
        printCurrent();
        std::cout << "\n";
        return;
    }
    int r = pos / N, c = pos % N;

    if (grid[r][c] != 0) {  // given clue, move on
        solve(pos + 1);
```

```
            return;
        }
        for (int v = 1; v <= 9; v++) {
            if (canPlace(r, c, v)) {
                grid[r][c] = v;
                solve(pos + 1);
                grid[r][c] = 0;  // backtrack
            }
        }
    }

    static void print(const int g[N][N]) {
        for (int r = 0; r < N; r++) {
            for (int c = 0; c < N; c++) {
                if (g[r][c] == 0) std::cout << "_ ";
                else std::cout << g[r][c] << " ";
            }
            std::cout << "\n";
        }
    }

    void printCurrent() const {
        for (int r = 0; r < N; r++) {
            for (int c = 0; c < N; c++) std::cout << grid[r][c] << " ";
            std::cout << "\n";
        }
    }
};

int main(int argc, char *argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; i++) files.push_back(argv[i]);
    } else {
        for (int i = 1; i <= 5; i++)
            files.push_back("puzzle" + std::to_string(i) + ".txt");
    }

    for (size_t i = 0; i < files.size(); i++) {
        Sudoku s(files[i]);
        if (s.isLoaded()) s.run();
```

```
    }
    return 0;
}
```

Correctness: The program was tested on puzzle1.txt through puzzle5.txt and matched solution1.txt expectations exactly, including the solution count of 1 or more depending on the puzzle. It does not crash on a missing file, a file with too few cells, or an invalid character, and instead prints an error message. Puzzles whose given clues already conflict print “No solution found”.

Execution Time: The search tries digits 1-9 in each blank cell, so the worst case is O(9^n), where n is the number of blank cells. Exponential, but each placement is checked against its row, column, and box first, so most branches are cut early. A check is a fixed 27 comparisons, so it costs O(1).

Space Complexity: The grid and the saved original puzzle are fixed 9×9 arrays, and the recursion is at most 81 numbers deep. That makes the space O(1).

Maintainability: Has a Sudoku class with small, descriptively named functions like canPlace, validStart, solve, load. Each function is generally commented, and the board size is a constant N. The 3 (inner box size) and 9 (maximum digits) are still hardcoded though. solve() also prints solutions directly instead of returning them, which makes it harder to reuse. Bad input makes an error message, so debugging is easier.

Deepseek Code:

```
//deepseek.cpp contained in file Joshua_Louis_Assignment_4
//C++ program that solves sudoku puzzles
//Creation Date: 11:05AM, Thursday, October 15, 2026
//Inputs: direct inputs with bash command "./deepseek puzzle1.txt" or just run
"./deepseek" for all 5 puzzles
/*Outputs: The program outputs the puzzle's file, puzzle itself, each solution, and
solutions
    found for every input.*/
//Collaborators: Deepseek - Free
//Note: None of the code below is original/human-made, it was all generated by Claude -
Deepseek - Free
//Author: Joshua Louis

#include <iostream>
#include <fstream>
#include <string>
#include <vector>

class SudokuSolver {
public:
    SudokuSolver(const std::string& filename) : filename_(filename) {
        grid_.resize(9, std::vector<char>(9, '_'));
        readPuzzle();
    }

    void solveAndPrint() {
        std::cout << "Puzzle file: " << filename_ << "\n";
        std::cout << "Puzzle:\n";
        printGrid(grid_);
        std::cout << "\n";

        solutions_.clear();
        solve(0, 0);

        if (solutions_.empty()) {
            std::cout << "No solution found\n";
        } else {
            for (size_t i = 0; i < solutions_.size(); ++i) {
                std::cout << "Solution " << (i + 1) << ":\n";
                printGrid(solutions_[i]);
```

```
                std::cout << "\n";
            }
            std::cout << "Solutions found: " << solutions_.size() << "\n";
        }
        std::cout << "----------------------------------------\n";
    }

private:
    std::string filename_;
    std::vector<std::vector<char>> grid_;
    std::vector<std::vector<std::vector<char>>> solutions_;

    void readPuzzle() {
        std::ifstream infile(filename_);
        if (!infile) {
            std::cerr << "Error opening file: " << filename_ << "\n";
            exit(1);
        }
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                infile >> grid_[r][c];
            }
        }
    }

    void printGrid(const std::vector<std::vector<char>>& g) const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                std::cout << g[r][c];
                if (c < 8) std::cout << ' ';
            }
            std::cout << '\n';
        }
    }

    bool isValid(const std::vector<std::vector<char>>& g, int row, int col, char num)
const {
        // Check row
        for (int c = 0; c < 9; ++c) {
            if (g[row][c] == num) return false;
        }
```

```
        // Check column
        for (int r = 0; r < 9; ++r) {
            if (g[r][col] == num) return false;
        }
        // Check 3x3 box
        int boxRow = (row / 3) * 3;
        int boxCol = (col / 3) * 3;
        for (int r = boxRow; r < boxRow + 3; ++r) {
            for (int c = boxCol; c < boxCol + 3; ++c) {
                if (g[r][c] == num) return false;
            }
        }
        return true;
    }

    void solve(int row, int col) {
        // If we've reached the end of the grid, we found a solution
        if (row == 9) {
            solutions_.push_back(grid_);
            return;
        }

        // Move to next cell
        int nextRow = row;
        int nextCol = col + 1;
        if (nextCol == 9) {
            nextRow = row + 1;
            nextCol = 0;
        }

        // If current cell is already filled, move on
        if (grid_[row][col] != '_') {
            solve(nextRow, nextCol);
            return;
        }


        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(grid_, row, col, num)) {
                grid_[row][col] = num;
                solve(nextRow, nextCol);
```

```
                grid_[row][col] = '_';
            }
        }
    }
};

int main() {
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto& file : puzzleFiles) {
        SudokuSolver solver(file);
        solver.solveAndPrint();
    }

    return 0;
}
```

Correctness: Similar to Claude, the program was tested on puzzle1.txt through puzzle5.txt and matched solution1.txt expectations exactly, including the solution count of 1 or more depending on the puzzle. It doesn't crash on a missing file, a file with too few cells, or an invalid character, and instead prints an error message. Puzzles whose given clues already conflict print "No solution found". There is no input validation for characters other than _ or 1–9, so a stray letter would be treated as a pre-filled cell and silently corrupt the search.

Time Complexity: Worst case complexity is O(9^n), similar to Claude, where n is the number of blank numbers.

Space Complexity: O(1) because the grid is a 9x9 array of characters. The dominant cost is `solutions_`, which stores a full 9×9 copy of the grid for every solution found: O(n) space, where n is the number of solutions.

Maintainability: Good overall structure and descriptive names, but error handling is minimal, file-open failure terminates the program, and invalid input is not reported. Not many included comments.

Selected AI:

Deepseek - Free Mode

Reason:

Deepseek's code is easier for me to read than Claude's.

What I did to improve:

Correctness: Prompted Deepseek to improve code by adding input validation.

Execution Time: Did not improve nor remove.

Space Complexity: Did not improve nor remove.

Maintainability: Added comments to each line. Prompted Deepseek to add error messages for invalid input.